МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ
з лабораторної роботи №2
з дисципліни "Основи Програмної Інженерії"

Виконав
ст. гр. ПЗПІ-25-3
Гладких.А.Р

Прийняв:
В.Гребенюк

Харків 2026

ЛАБОРАТОРНА РОБОТА № 2

МОДЕЛЮВАННЯ ПРОГРАМНОЇ СИСТЕМИ ЗАСОБАМИ UML (UNIFIED MODELING LANGUAGE)

2.1 Мета роботи

Навчитися трансформувати функціональні вимоги до програмного забезпечення у візуальні моделі UML. Оволодіти інструментарієм проектування структури та динаміки системи (Use Case, Class, Sequence diagrams). Продемонструвати навички аналізу предметної області та побудови матриць відповідності для контролю покриття вимог проєктними рішеннями.

2.2 Хід роботи

2.2.1 Предметна область

DeltaPhys - платформа для пошуку партнерів та професійного нетворкінгу в екстремальних видах спорту.

2.2.2 Функціональні вимоги

FR-01: Модуль автентифікації повинен забезпечувати реєстрацію нових користувачів через електронну пошту.

FR-02: Функціонал системи має дозволяти формування пошукових запитів у вигляді анкет-сесій для нетворкінгу.

FR-03: Система повинна генерувати публічний шар карти з геомітками запланованих спортивних подій.

FR-04: Програмне забезпечення має надавати статус «верифіковано» користувачам після перевірки їхніх даних.

FR-05: У системі має бути реалізована можливість обміну текстовими повідомленнями між учасниками конкретного виїзду.

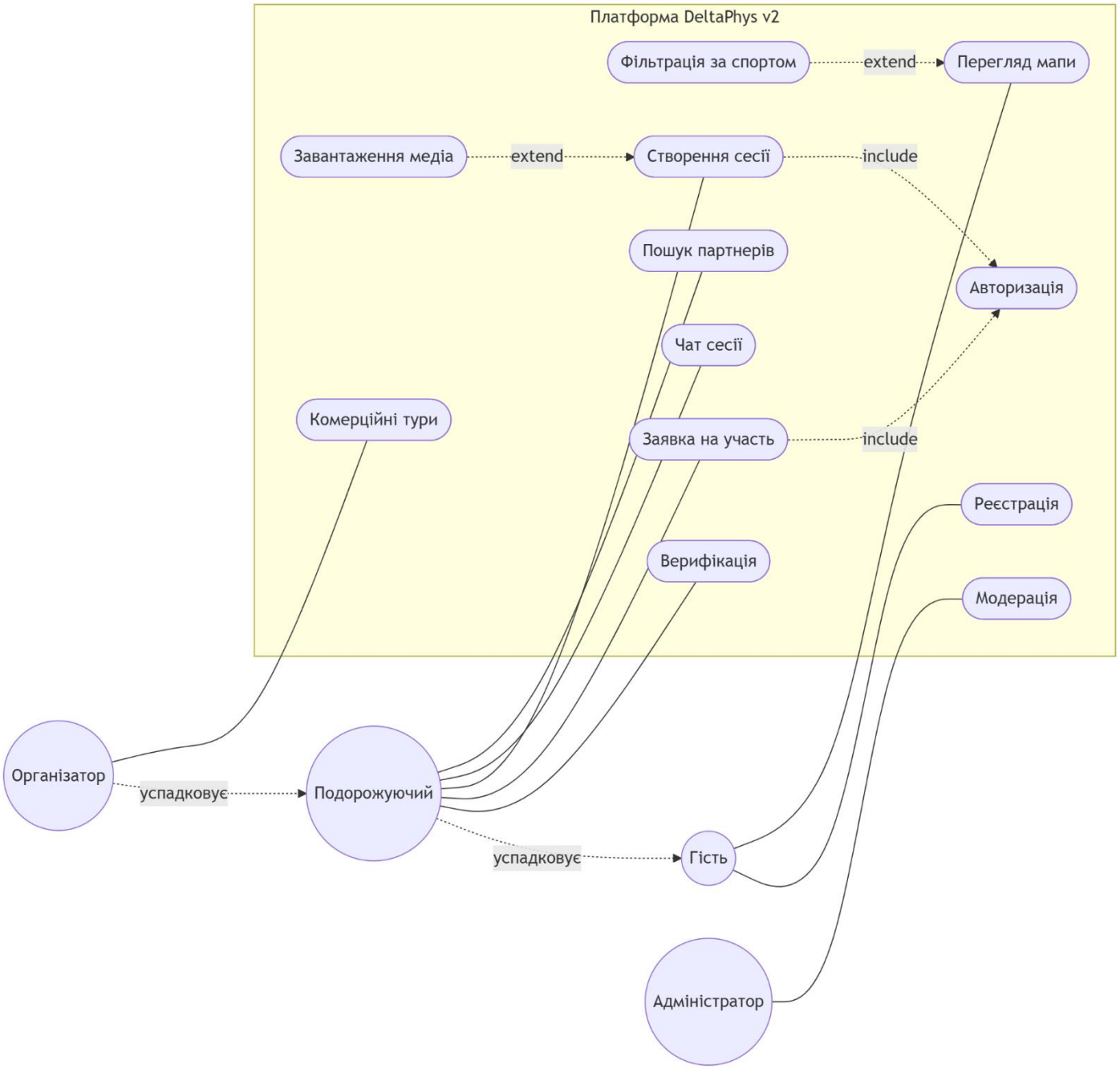
FR-06: Система повинна дозволяти фільтрацію подій за видом спорту

2.2.3 Діаграма прецедентів

Код

```
flowchart LR
    subgraph Actors [ ]
        direction TB
        G((Гість))
        T((Подорожуючий))
        O((Організатор))
        A((Адміністратор))
    end

    subgraph System [Платформа DeltaPhys v2]
        direction TB
        UC1([Перегляд мапи])
        UC11([Фільтрація за спортом])
        UC2([Реєстрація])
        UC_Auth([Авторизація])
        UC3([Пошук партнерів])
        UC4([Чат сесії])
        UC5([Створення сесії])
        UC_Upload([Завантаження медіа])
        UC6([Заявка на участь])
        UC7([Верифікація])
        UC8([Комерційні тури])
        UC10([Модерація])

        UC5 -.->|include| UC_Auth
        UC6 -.->|include| UC_Auth
        UC11 -.->|extend| UC1
        UC_Upload -.->|extend| UC5
    end

    G --- UC1
    G --- UC2

    T --- UC3
    T --- UC4
    T --- UC5
    T --- UC6
    T --- UC7

    O --- UC8
    A --- UC10

    T -.->|успадковує| G
    O -.->|успадковує| T

    style Actors fill:none,stroke:none
```

Діаграма



Виконав: Гладких

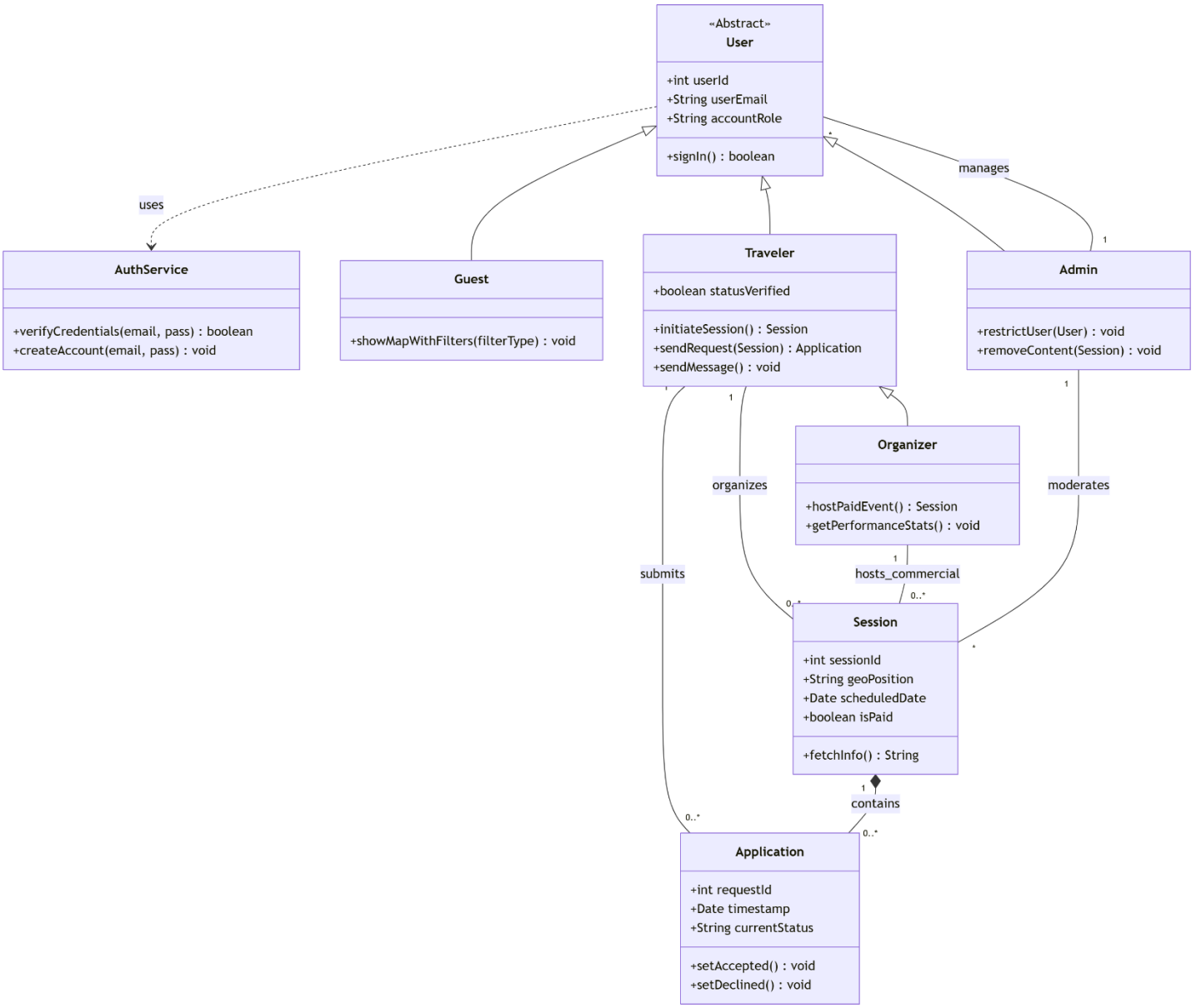
2.2.4 Діаграма класів

Код

```
classDiagram
    class AuthService {
        +verifyCredentials(email, pass) boolean
        +createAccount(email, pass) void
    }

    class User {
        <<Abstract>>
        +int userId
        +String userEmail
        +String accountRole
        +signIn() boolean
    }

    class Guest {
        +showMapWithFilters(filterType) void
    }

    class Traveler {
        +boolean statusVerified
        +initiateSession() Session
        +sendRequest(Session) Application
        +sendMessage() void
    }

    class Organizer {
        +hostPaidEvent() Session
        +getPerformanceStats() void
    }

    class Admin {
        +restrictUser(User) void
        +removeContent(Session) void
    }

    class Session {
        +int sessionId
        +String geoPosition
        +Date scheduledDate
        +boolean isPaid
        +fetchInfo() String
    }

    class Application {
        +int requestId
        +Date timestamp
        +String currentStatus
        +setAccepted() void
        +setDeclined() void
```

```
}

User <|-- Guest
User <|-- Traveler
User <|-- Admin
Traveler <|-- Organizer

User ..> AuthService : uses

Traveler "1" -- "0..*" Session : organizes
Traveler "1" -- "0..*" Application : submits
Organizer "1" -- "0..*" Session : hosts_commercial
Session "1" *-- "0..*" Application : contains
Admin "1" -- "*" User : manages
Admin "1" -- "*" Session : moderates
```

Діаграма



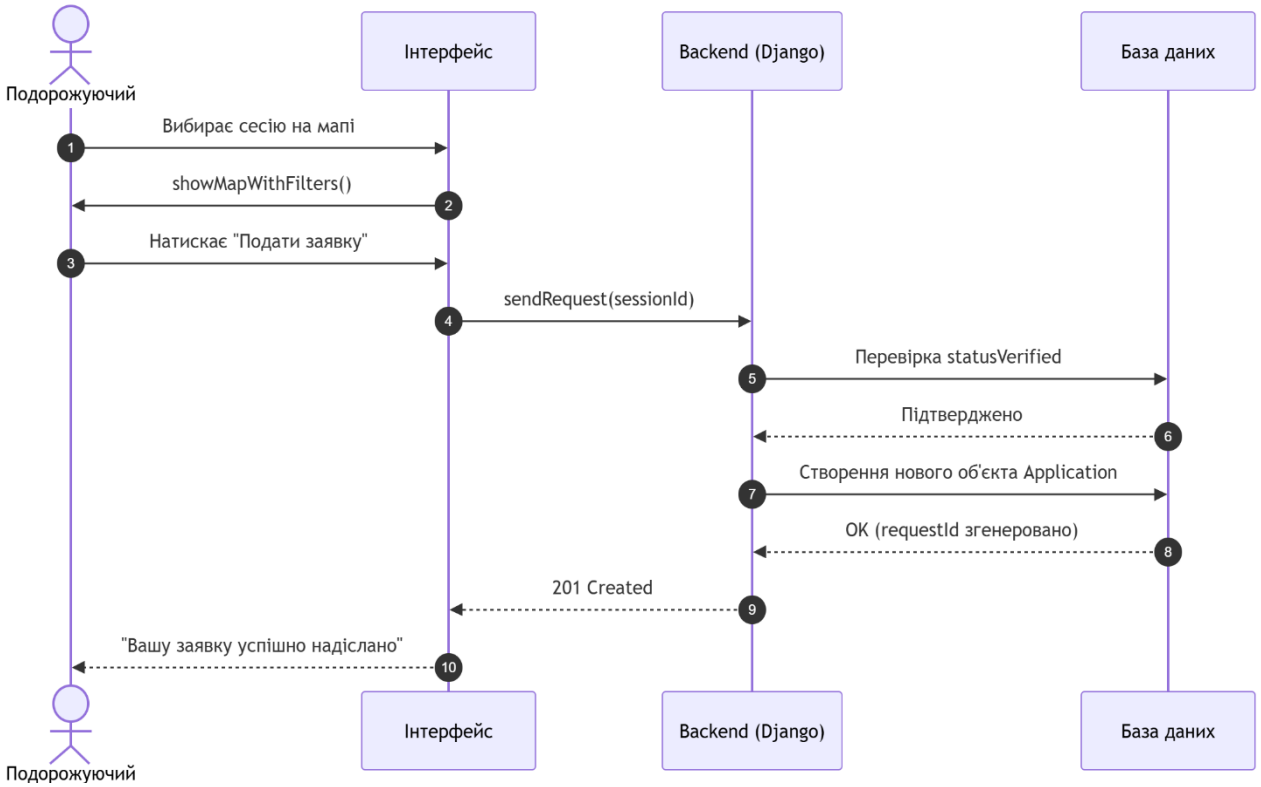
Виконав: Гладких

2.2.5 Діаграма послідовності

Код

```
sequenceDiagram
    autonumber
    actor T as Подорожуючий
    participant UI as Інтерфейс
    participant S as Backend (Django)
    participant DB as База даних

    T->>UI: Вибирає сесію на мапі
    UI->>T: showMapWithFilters()
    T->>UI: Натискає "Подати заявку"
    UI->>S: sendRequest(sessionId)
    S->>DB: Перевірка statusVerified
    DB-->>S: Підтверджено
    S->>DB: Створення нового об'єкта Application
    DB-->>S: OK (requestId згенеровано)
    S-->>UI: 201 Created
    UI-->>T: "Вашу заявку успішно надіслано"
```

Діаграма



Виконав: Гладких

2.2.6 Матриця трасовності

Вимога	Use Case	Класи	Sequence
FR-01: Реєстрація через email	UC2: Реєстрація	AuthService, User	
FR-02: Пошукові запити (анкети)	UC5: Створення сесії	Traveler, Session	
FR-03: Публічна карта	UC1: Перегляд мапи	Guest, Session	
FR-04: Статус «верифіковано»	UC7: Верифікація	Traveler (атрибут `statusVerified`)	Перевірка `statusVerified`
FR-05: Обмін повідомленнями	UC4: Чат сесії	Traveler (метод `sendMessage`)	
FR-06: Фільтрація за спортом	UC11: Фільтрація	`Guest` (метод `showMapWithFilters`)	showMapWithFilters()